# 作业：软件测试

## 作业目标

为项目编写测试用例，建立质量保障体系。**根据你在团队中的分工**，完成对应角色的测试任务。

---

## 后端成员任务

### 1. 单元测试（业务逻辑）

对核心 Service / 工具函数编写单元测试，**使用 Mock 隔离数据库和外部 API**：

```python
# Python 示例：测试业务逻辑 + Mock 数据库
from unittest.mock import patch

@patch('service.db.query')
def test_get_user_returns_name(mock_query):
    mock_query.return_value = {"id": 1, "name": "Alice"}
    result = get_user(1)
    assert result["name"] == "Alice"
    mock_query.assert_called_once_with(id=1)
```

```java
// Java 示例：Mock Repository
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository repo;
    @InjectMocks OrderService service;

    @Test
    void createOrder_savesAndReturnsId() {
        when(repo.save(any())).thenReturn(new Order(42L));
        assertEquals(42L, service.createOrder(new OrderRequest()));
    }
}
```

### 2. API / 接口测试

测试接口的正常响应、参数校验和异常处理：

```python
# pytest + Flask/FastAPI 测试客户端
def test_login_success(client):
    res = client.post('/api/auth/login',
                      json={'email': 'a@b.com', 'password': '123456'})
    assert res.status_code == 200
    assert 'token' in res.json()

def test_login_wrong_password(client):
    res = client.post('/api/auth/login',
                      json={'email': 'a@b.com', 'password': 'wrong'})
    assert res.status_code == 401

def test_login_missing_email(client):
    res = client.post('/api/auth/login', json={'password': '123456'})
    assert res.status_code == 422   # 参数校验失败
```

### 3. 后端数量要求

| 测试类型 | 最低数量 |
|---|---|
| 单元测试（含 Mock） | ≥ 8 个 |
| API 接口测试 | ≥ 6 个（含异常情况） |
| 核心模块覆盖率 | > 60% |

---

## 前端成员任务

### 1. 组件测试

测试组件的**渲染输出**和**用户交互行为**，关注用户视角而非实现细节：

```typescript
// React Testing Library 示例
import { render, screen, fireEvent } from '@testing-library/react';
import LoginForm from './LoginForm';

test('点击登录按钮触发提交', () => {
  const onSubmit = jest.fn();
  render(<LoginForm onSubmit={onSubmit} />);

  fireEvent.change(screen.getByLabelText('邮箱'), {
    target: { value: 'a@b.com' }
  });
  fireEvent.click(screen.getByRole('button', { name: /登录/i }));
```

```
  expect(onSubmit).toHaveBeenCalledWith({ email: 'a@b.com' });
});

test('密码为空时显示错误提示', () => {
  render(<LoginForm onSubmit={jest.fn()} />);
  fireEvent.click(screen.getByRole('button', { name: /登录/i }));
  expect(screen.getByText('密码不能为空')).toBeInTheDocument();
});
```

### 2. Mock API 请求

用 Mock 替代真实网络请求，测试加载态、成功态和失败态：

```
// Jest mock axios
jest.mock('axios');

test('加载成功后显示用户名', async () => {
  axios.get.mockResolvedValue({ data: { name: 'Alice' } });
  render(<UserProfile userId={1} />);

  expect(screen.getByText('加载中...')).toBeInTheDocument();
  await waitFor(() =>
    expect(screen.getByText('Alice')).toBeInTheDocument()
  );
});

test('加载失败时显示错误信息', async () => {
  axios.get.mockRejectedValue(new Error('Network Error'));
  render(<UserProfile userId={1} />);
  await waitFor(() =>
    expect(screen.getByText('加载失败')).toBeInTheDocument()
  );
});
```

### 3. 前端数量要求

| 测试类型 | 最低数量 |
|---|---|
| 组件渲染 / 交互测试 | ≥ 8 个 |
| Mock API 测试（含失败场景） | ≥ 4 个 |
| 核心组件覆盖率 | > 50% |

---

# 所有成员共同要求

## 覆盖率报告

```
# Python
pytest --cov=app --cov-report=html --cov-report=term-missing

# JavaScript / TypeScript
npm test -- --coverage
```

## AI 辅助测试（加分）

使用 AI 工具（Cursor / GitHub Copilot 等）辅助生成测试用例，在贡献说明中记录使用的 Prompt 和修改过程。

## 在 README 中展示覆盖率徽章

接入 Codecov 后，前后端分别上传覆盖率报告，在 `README.md` 顶部用两个徽章分别展示：

```
# .github/workflows/test.yml（示例）
- name: 上传后端覆盖率
  uses: codecov/codecov-action@v4
  with:
    token: ${{ secrets.CODECOV_TOKEN }}
    files: backend/coverage.xml
    flags: backend           # 用 flag 区分前后端

- name: 上传前端覆盖率
  uses: codecov/codecov-action@v4
  with:
    token: ${{ secrets.CODECOV_TOKEN }}
    files: frontend/coverage/lcov.info
    flags: frontend
```

在 `README.md` 中插入对应徽章：

```
[![Backend  Coverage](https://codecov.io/gh/<用户名>/<仓库名>/branch/main/graph/badge.svg?flag=backend)](https://codecov.io/gh/<用户名>/<仓库名>)
[![Frontend  Coverage](https://codecov.io/gh/<用户名>/<仓库名>/branch/main/graph/badge.svg?flag=frontend)](https://codecov.io/gh/<用户名>/<仓库名>)
```

效果：README 顶部同时显示后端和前端两个独立的覆盖率数字。

---

## 目录结构

```
project/
├── backend/
│   └── tests/
│       ├── conftest.py          # pytest fixtures
│       ├── test_api.py          # 接口测试
│       └── test_services.py     # 业务逻辑单元测试
├── frontend/
│   └── src/
│       └── __tests__/           # 或 *.test.jsx
│           ├── LoginForm.test.jsx
│           └── UserProfile.test.jsx
└── docs/
    └── contributions/
        └── 08-testing/
            ├── zhangsan.md
            └── lisi.md
```

---

## 提交内容

### Git 仓库

**后端成员：**

1. `backend/tests/` 目录下的测试代码
2. 覆盖率配置文件（`pytest.ini` / `pyproject.toml` 等）
3. `docs/contributions/08-testing/你的名字.md`

**前端成员：**

1. `frontend/src/__tests__/` 或 `*.test.jsx/tsx` 测试文件
2. 覆盖率配置文件（`jest.config.js` / `vitest.config.ts` 等）
3. `docs/contributions/08-testing/你的名字.md`

### 学习通截图

1. **Git 提交记录** — `git log --author="YOUR_NAME" --oneline` 截图
2. **测试运行结果** — 全部测试通过的终端截图
3. **覆盖率报告** — 显示覆盖率达标的截图

> 💡 确保 `npm test` 或 `pytest` 可在组员电脑上直接运行，无需额外配置。

**截止时间**：下次课前

---

## 个人贡献说明模板

文件路径：`docs/contributions/08-testing/你的名字.md`

```
# 软件测试贡献说明

姓名：XXX　学号：XXX　角色：前端 / 后端　日期：XXXX-XX-XX

## 完成的测试工作

### 测试文件
- `tests/test_xxx.py` 或 `src/__tests__/Xxx.test.jsx`

### 测试清单
- [ ] 正常情况测试（X 个）
- [ ] 边界 / 异常情况测试（X 个）
- [ ] Mock 使用（数据库 / API / 组件外部依赖）

### 覆盖率
- 核心模块覆盖率：XX%

### AI 辅助（如有）
- 使用工具：Cursor / GitHub Copilot / 其他
- Prompt 示例：XXX
- AI 生成 + 人工修改的测试数量：XX 个

## PR 链接
- PR #X: https://github.com/xxx/xxx/pull/X

## 遇到的问题和解决
1. 问题：XXX → 解决：XXX

## 心得体会
（简要说明收获）
```

---

## 评分标准

| 项目 | 分值 | 说明 |
|---|---|---|
| 测试数量 | 25% | 达到角色对应的最低数量要求 |
| 测试质量 | 25% | 覆盖正常 / 边界 / 异常场景，Mock 使用合理 |
| 覆盖率 | 20% | 达到角色对应的覆盖率要求 |

| 项目 | 分值 | 说明 |
|---|---|---|
| 测试通过率 | 15% | 全部测试通过，可在他人机器运行 |
| 个人贡献说明 | 15% | 内容清晰，有截图佐证 |

**额外加分项**

- 使用 AI 辅助并记录完整 Prompt 过程
- 覆盖率超过 80%
- 在 GitHub Actions 中集成测试自动运行 + Codecov 徽章
- 编写 E2E 测试（Playwright / Espresso / Flutter integration_test）